

DyGNN: Algorithm and Architecture Support of Dynamic Pruning for Graph Neural Networks

Cen Chen[†], Kenli Li^{*,†}, Xiaofeng Zou[†], Yangfan Li[†],

^{*}Corresponding Author

[†]College of Information Science and Engineering, Hunan University, China
{chencen, lkl, zouxiaofeng, yangfanli}@hnu.edu.cn

Abstract—Recently, graph neural networks (GNNs) have achieved great success for graph representation learning tasks. Enlightened by the fact that numerous message passing redundancies exist in GNNs, we propose DyGNN, which speeds up GNNs by reducing redundancies. DyGNN is supported by an algorithm and architecture co-design. The proposed algorithm can dynamically prune vertices and edges during execution without accuracy loss. An architecture is designed to support dynamic pruning and transform it into performance improvement. DyGNN opens new directions for accelerating GNNs by pruning vertices and edges. DyGNN gains average $2\times$ speedup with accuracy improvement of 4% compared with state-of-the-art GNN accelerators.

Index Terms—Graph Neural Networks, Software-Hardware Co-Design, Neural Network Pruning

I. INTRODUCTION

Recently, graph neural networks (GNNs) [5], [8], [14], which extend traditional deep neural networks (DNNs) to handle irregularly structured graphs, have achieved great success across various applications, such as computer vision, natural language processing and recommendation system [15]. The majority of existing GNNs utilize neighborhood messages passing mechanism based on neural networks that updates target vertex representations by aggregating feature messages from neighboring source vertices [12], [15], as shown in Fig. 1(a). Several message passing operations for the graphs are conducted iteratively to make vertex representations can receive messages from multi-hop neighbors. Generally, computations in a message passing operation can be abstracted into three key stages [1], [3], [12]: *EdgeUpdate*, *Aggregate* and *VertexUpdate*. To accelerate the computations of GNNs, several customized accelerators [1], [4], [7], [10], [17] are proposed based on the computing model, achieving much higher performance and lower energy consumption than GPUs and existing DNN accelerators for GNNs.

The computing abstraction in message passing includes several time-consuming operations for each vertex and each edge in GNNs, as shown in Fig. 1(b). Worse, the real-world

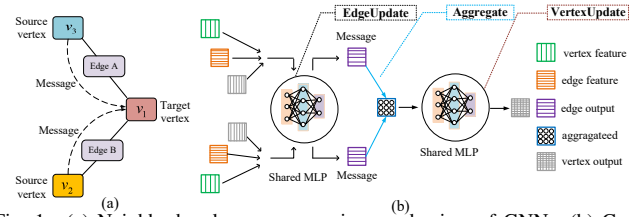


Fig. 1. (a) Neighborhood message passing mechanism of GNNs. (b) Computing abstraction of neighborhood message passing mechanism.

graphs are composed by numerous vertices and edges. This motivates us to answer a question: *Can we prune vertices and edges to speed up execution of GNNs without losses of accuracy?*

Through analyzing, we observe that there are numerous message passing redundancies for vertices and edges in GNNs and redundant message passing execution may even decrease the accuracy on the contrary. Enlightened by these observations, we propose an algorithm that prunes vertices and edges dynamically during execution, maintaining or even improving the original accuracy on popular benchmark datasets.

Then we design a novel architecture, termed as DyGNN, to support pruning vertices and edges dynamically and transform the dynamic reducing mechanism into performance improvement. There are three main types of engines in DyGNN: *Aggregator*, *Updater* and *Pruner*. Among those, *Pruner* is in charge of pruning vertices or edges based on our proposed pruning algorithm. *Aggregator* is responsible for conducting aggregation. In *Updater*, we build two multi-granular systolic arrays to perform matrix-vector multiplication (MVM) for edges and vertices separately. These *Updaters* can also be combined together to perform MVM for vertices. To efficiently fuse the phase-by-phase execution, we propose a duplication-free mechanism in *Pruner* which can significantly reduce data duplication to improve efficiency by fully exploiting data reuse. To support representative variants of GNNs, we design a configurable pipelined architecture that can support different execution flow by control command. In summary, we have the following contributions:

- We analyze the behavior of the message aggregation mechanism in GNN and demonstrate the feasibility of DyGNN.
- We propose an algorithm that prunes vertices and edges dynamically during execution, while maintaining or even improving accuracies on popular benchmark datasets.

This work was partially funded by the National Key Research and Development Program of China under Grant 2018YFB0203800, by the National Natural Science Foundation of China under Grant 61902120, by the National Outstanding Youth Science Program of National Natural Science Foundation of China under Grant 61625202, by the Postdoctoral Science Foundation of China under Grant 2019M662768 and Grant 2019TQ0086, and in part by the Natural Science Foundation of Hunan Province under Grant 2020JJ5083, and by the Cultivation of Shenzhen Excellent Technological and Innovative Talents (Ph.D. Basic Research Started) RCB520200714114943014. We thank Dr. Tao Li for his significant contribution to this work.

- We propose a novel architecture design to support efficient dynamic vertices and edges pruning.
- We show that DyGNN is a flexible architecture that can support various GNNs with pre-defined configurations.

Different from conventional pruning methods for parameters or activations in DNNs, DyGNN opens new opportunities of pruning vertices and edges for the acceleration of GNNs.

II. MOTIVATION AND ALGORITHM DESIGN

A. Computing Abstraction of Graph Neural Networks

The current vigorous development of GNNs has derived many successful variants, such as GCN [8], GAT [14]. These variants broadly follow the message passing strategy [15], which updates vertex features by gathering feature messages from neighboring vertices as shown in Fig. 1(a). The computing process of a message passing layer usually contains three key operations as follows [12]:

- *EdgeUpdate* computes the messages for each edge by utilizing Multilayer Perceptron (MLP), which is usually expressed by matrix-vector multiplication (MVM).
- *Aggregate* gathers the neighboring messages for each target vertex and reduces it to a single feature vector.
- *VertexUpdate* combines the old vertex features of each target vertex and the aggregated features to produce new vertex features of each target vertex by utilizing MLP.

In general, GNNs fall into two categories [12]: GNNs without *EdgeUpdate* and GNNs with *EdgeUpdate*. For the first one, a typical algorithm is GCN [8], which only involves an *Aggregation* phase for neighbors of each vertex and a *Update* phase for each vertex based on the aggregated features. For the second one, a typical algorithm is GAT [14], which contains an additional phase before *Aggregation*: the MLP is utilized to update the message for each edge. Tab. I lists the notations used in this paper.

Table I. Notations and Explanations.

Notation	Explanation	Notation	Explanation
G	graph $G = (V, E)$	V	vertices of G
E	edges of G	$e(u, v)$	edge between vertex u and v
h_v	feature vector of vertex v	a_v	aggregation feature vector of v
W_{edge}	edge weight parameters	W_{vertex}	vertex weight parameters
$N(v)$	v ' neighbor set	λ	feature smoothness
$es(u, v)$	smoothness of $e(u, v)$	$ns(v)$	smoothness of vertex v

B. Feasibility of DyGNN

Computational Redundancy in GNNs. Recent studies [9], [11], [16] have found that the over-smoothing issue greatly affects the quality of message passing in GNNs. An example is illustrated in Fig. 2(a) which shows that after stacking deep layers of GNNs (*i.e.*, GCN and GAT), the performance of GNNs on PubMed dataset exhibits much degradation. We argue that the reason for this phenomenon is the introduction of redundant message passing. Fig. 2(b) illustrates the number of vertices that turn from misclassification to correct classification and the vertices which turn from correct classification to misclassification. After some layers, the number of vertices that turn from correct classification to misclassification increases quickly which leads to poor average accuracy. This implies that different vertices may need different messaging passing layers. Nevertheless, GNNs performs the same message passing operations on all vertices and edges, which introduces redundant message passing and makes vertex representations

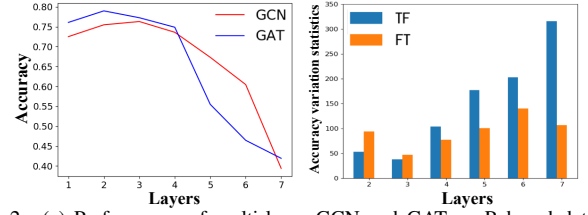


Fig. 2. (a) Performance of multi-layer GCN and GAT on Pubmed dataset. (b) Accuracy variation statistics. TF means that the vertices turn from correct classification to misclassification, and FT means that the vertices turn from misclassification to correct classification.

of different classes indistinguishable. We analyze this issue from two aspects: vertex redundancy and edge redundancy.

Vertex Redundancy. In GNNs, some vertices may converge in early layers and can produce correct predictions, while the remaining misclassified vertices may require more message passing processes to obtain optimal representations. If more messages passing layers are stacked, redundant computations are introduced into the vertices that have converged already. These convergent vertices will lose their specific information due to redundant propagation, turning from correct classification to misclassification.

Edge Redundancy. Furthermore, we argue that there is a large quantity of edge redundancy during the neighborhood aggregation phase. We think that not all neighbors of a vertex contain task-related information in the neighborhood aggregation phase. For instance, when the vertex is much similar to the neighborhood, the neighborhood may not contribute extra information to the vertex. At this time, these edge connections would lead to redundant computing.

C. Design of GNN Algorithm with Dynamic Pruning

To verify the above analysis, we first introduce a unified quantitative metric: *feature smoothness* λ , which estimates which vertices or edges need pruning. Formally, given the feature vector of vertex u and vertex v , we define *feature smoothness* with their Manhattan distance:

$$\lambda_{uv} = \frac{\|h_u - h_v\|_1}{d}, \quad (1)$$

where d is the dimension of the feature. A smaller λ_{uv} indicates that the feature vectors h_u and h_v are similar, and that the features are smoother.

GNN Algorithm with Dynamic Pruning. Algorithm 1 shows the proposed GNN algorithm with dynamic pruning based on the smoothness metric. We propose two pruning operations: *EdgePrune* and *VertexPrune*. For each vertex $v \in V$, the process of *EdgePrune* is conducted for each edge of v . If the edge is reserved, *EdgeUpdate* is conducted for the features of this pair and then the updated features are aggregated. After all the neighbors of v are aggregated, the *NodePrune* operation executes. Finally, for the reserved nodes, *VertexUpdate* is performed to obtain the new node representations. Consequently, pruned nodes and edges are not considered in the following message passing layers. Details of the *EdgePrune* and *VertexPrune* are described in the following.

EdgePrune. The *EdgePrune* process aims to prune the redundant edges of which the connected vertices are similar, and block messages passing through this edge to eliminate the

Algorithm 1 GNN algorithm with dynamic pruning.

```

1: for  $k$  max number of layers  $K$  do
2:   for each  $v \in V$  do
3:      $\triangleleft$  EdgePrune
4:     for each  $u$  in the neighbor list of  $v$  do
5:        $es_{(u,v)} \leftarrow \text{SmoothnessCompute}(h_u, h_v)$ ;
6:       if  $es \leq \text{threshold}$  then
7:         Prune  $edge(u, v)$ 
8:       end if
9:     end for
10:     $\triangleleft$  EdgeUpdate (Optional)
11:    for each reserved  $u$  in the neighbor list of  $v$  do
12:       $e_{(u,v)} \leftarrow \text{EdgeUpdate}(h_v^{k-1}, h_u^{k-1}, W_{edge})$ ;
13:    end for
14:     $\triangleleft$  Aggregation
15:    for each reserved  $u$  in the neighbor list of  $v$  do
16:       $a_v^k \leftarrow \text{Aggregate}(a_v^k, e_{(u,v)}, h_u^{k-1})$ ;
17:    end for
18:     $\triangleleft$  VertexPrune
19:     $ns_{(u)} \leftarrow \text{SmoothnessCompute}(a_v^k, h_v^{k-1})$ ;
20:    if  $ns \leq \text{threshold}$  then
21:      Prune  $v$ 
22:    else
23:      Reserve  $v$ 
24:    end if
25:     $\triangleleft$  VertexUpdate
26:    if  $v$  is reserved then
27:       $h_v^k \leftarrow \text{VertexUpdate}(a_v^k, h_v^{k-1}, W_{vertex})$ ;
28:    end if
29:  end for
30: end for

```

Edge Redundancy. Given an $edge(u, v)$, we utilize the feature vectors of vertices u and v to calculate feature smoothness $es_{(u,v)}$. If $es_{(u,v)}$ is smaller than a defined threshold, it means that u and v are similar, and $edge(u, v)$ would be pruned.

VertexPrune. The idea of *VertexPrune* is that if a vertex is considered as converged, it can be pruned to stop aggregating messages to eliminate the *Vertex Redundancy*. We compute the feature smoothness ns_v with input vertex features h_v^{k-1} and aggregated features a_v^k . If ns_v is smaller than a pre-defined threshold, v will be pruned. Note that the pruned vertex can still pass its own features to other vertices.

Effects of Dynamic Pruning. Tab. II and Tab. III show the effect of pruning edges and vertices in different GCN layers. We can observe that our method can not only prune many edges and vertices, but also improve the accuracy under different layers, which effectively alleviates the problem of computational redundancy.

Table II. Experimental results of pruning edges on Pubmed dataset.

Method	Layers	Accuracy		Threshold	Pruned Edges
		Original	Ours		
GCN	1	73.3	72.4	0.18	10170
	3	77.2	77.6	0.15	15444
	5	53.6	56.1	0.19	23642
	7	29.4	37.5	0.2	8890

Table III. Experimental results of pruning vertices on Pubmed dataset.

Method	Layers	Accuracy		Threshold	Pruned Vertices
		Original	Ours		
GCN	2	77.0	77.6	0.5	531
	3	77.2	77.9	0.5	855
	5	53.6	78.0	0.5	1772
	7	29.4	78.0	0.5	3047

III. ARCHITECTURE DESIGN OF DYGNN

DyGNN is an accelerator architecture for low-latency GNN inference that can support efficient dynamic pruning for vertices and edges during execution. Moreover, DyGNN can be adapted to support the two main existing categories of GNNs.

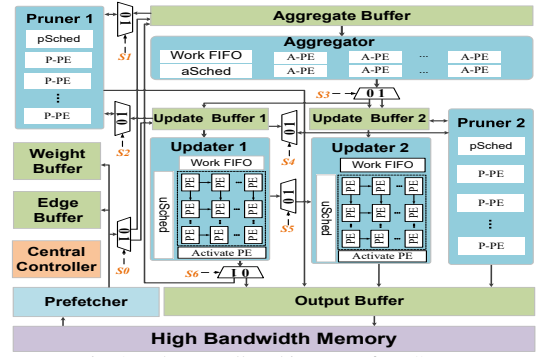


Fig. 3. The overall architecture of DyGNN.

A. Configurable Pipelined Architecture

Architecture Overview. Based on the proposed dynamic pruning algorithm, Fig. 3 illustrates the architecture of DyGNN. It contains three kinds of core execution engines: *Pruner*, *Aggregator*, and *Updater*. To reduce the latency of data access, we employ on-chip buffers to cache various data and intermediate results to improve data reuse. To hide the access latency, almost all buffers adopt double-buffering technology to allow overlapping execution of different operations.

In these engines, some schedulers are designed to assign the processing workloads. There are two *Pruners* (*Pruners 1* and *Pruners 2*) to support dynamic pruning of edges and vertices respectively. Among them, *Pruner 1* is adopted for *EdgePrune*, while *Pruner 2* is for *VertexPrune*. These two *Pruners* utilize the same pruning elements (P-PEs) for pruning. We equip DyGNN with two *Updaters* for feature transformation. In particular, *Updater 1* is for *EdgeUpdate*, and *Updater 2* is for *VertexUpdate*. In order to improve the processing parallelism and data reuse, we adopt the well-known systolic array [6] for MVM operations. After that, an activation operation is performed by *Activate* PEs to produce new feature vectors. Nevertheless, some GNNs do not have *EdgeUpdate* phase. To tackle this, the *Updaters* support multi-granularity usage, and these two *Updaters* can be combined together for *VertexUpdate* to support GNNs without *EdgeUpdate*. *Aggregator* is designed to execute the aggregation operation by iteratively accessing vertices and edges cached in the *Aggregate Buffer* by aggregating elements (A-PEs), and aggregated vertex features are saved in the *Update Buffer*.

To reduce on-chip memory duplication operations, we adopt a duplication-free scheme for pruning. After pruning, rather than duplicating data, the addresses of the reserved edges and vertices are pushed into *Work FIFO* in *Aggregator* or *Updaters* for execution.

DyGNN can flexibly configure the pipelined execution flow of these engines to support various GNNs. It consists of a *Central Controller* and seven micro-configurable *switches* ($S0$ - $S6$). The *Central Controller* can issue control commands asynchronously to manage all configurable *switches*. These micro-configurable switches can be configured according to the control command to match the target execution flow.

Flexible Pipelined Execution Flow. A *Prefetcher* is firstly utilized to explicitly prefetch the graph data and parameters from High Bandwidth Memory (HBM). Edges are cached in *Edge*

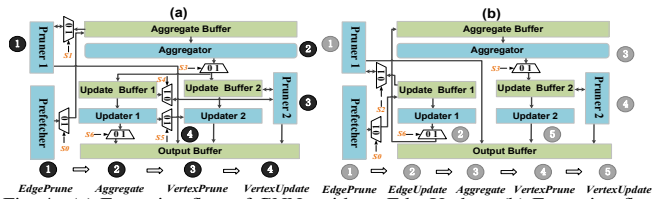


Fig. 4. (a) Execution flow of GNNs without EdgeUpdate. (b) Execution flow of GNNs that have EdgeUpdate.

Buffer, and weight parameters are cached in *Weight Buffer*. Note that for GNNs without *EdgeUpdate*, the *Prefetcher* stores the vertex features in *Aggregate Buffer*, and then *Pruner 1* executes *EdgePrune* for the data in *Aggregate Buffer*. For GNNs with *EdgeUpdate*, the *Prefetcher* stores the vertex features in *Updater Buffer 1*, and then *Pruner 1* executes *EdgePrune* for the data in *Updater Buffer 1*. The following execution flow for two main categories of GNNs is as follows.

Table IV. Configuration of switches. “-” means that the control signal without output.

Switches	Input	Control	Output
S0	<i>Prefetcher</i>	0:1	<i>Aggregate Buffer</i> : <i>Update Buffer 1</i>
S1	<i>Aggregate Buffer</i>	0:1	<i>Pruner 1</i> : -
S2	<i>Update Buffer 1</i>	0:1	-: <i>Pruner 1</i>
S3	<i>Aggregator</i>	0:1	<i>Update Buffer 1</i> &2: <i>Update Buffer 2</i>
S4	<i>Update Buffer 1</i>	0:1	<i>Pruner 2</i> : -
S5	<i>Updater 1</i>	0:1	<i>Updater 2</i> : -
S6	<i>Updater 1</i>	0:1	<i>Output Buffer</i> : <i>Aggregate Buffer</i>

Fig. 4(a) depicts the execution flow of GNNs without *EdgeUpdate*, which contains four pipelined phases: ① *EdgePrune*, which prunes neighbors of vertices in the *Aggregate Buffer* utilizing *Pruner 1*. ② *Aggregate*, which utilizes the *Aggregator* to perform aggregate operations on the reserved vertex neighbors of each vertex. ③ *VertexPrune*, which adopts the *Pruner 2* to prune the vertices in the *Update Buffer 2*. ④ *VertexUpdate*, which assembles two *Updaters* to update the remaining vertices.

Fig. 4(b) depicts the execution flow of GNNs with *EdgeUpdate*, which have an *EdgeUpdate* phase before *Aggregation*. Therefore, the execution dataflow needs to be re-configured, which is split into five phases: ① *EdgePrune*, ② *EdgeUpdate*, ③ *Aggregate*, ④ *VertexPrune*, and ⑤ *VertexUpdate*. In this model, the two *Updaters* are configured to perform *EdgeUpdate* and *VertexUpdate* respectively.

The configuration of each switch to support these pipelined execution flow is shown in Tab. IV. Signal of control can be configured as 0 for GNNs without *EdgeUpdate*, while 1 for GNNs with *EdgeUpdate*.

Inter-layer Execution. GNN models usually stack multiple layers according to graph structures. In our design, pruned results would modify the graph topology and are written into the *Output Buffer* for the execution of subsequent layers, making pruned vertices and edges not be considered in subsequent message passing layers, saving off-chip memory access cost.

B. Design of Pruner

Duplication-free Pruning Scheme. A natural implementation of *Pruner* is to iteratively accesses the data in the buffer, and then save the retained data in on-chip memory after pruning. After that, execution engines access the pruned results to perform corresponding operations. In this manner, a large quantity of on-chip memory duplication operations would be

introduced.

To tackle this issue, we propose a duplication-free pruning scheme. Fig. 5 illustrates the cooperation steps of *Pruner* and *Aggregator*. (1) **Prune**: *Pruner* would generate the addresses of the reserved vertices after pruning. (2) **Push work**: Then the generated vertex addresses are pushed into the *Work FIFO*. (3) **Aggregate**: The pruning scheduler (*aSched*) fetches addresses in the *Work FIFO*, and then input the features according to the addresses for aggregation. This approach can avoid data duplication cost and enable more data reuse.

Architecture Support of Pruning. Fig. 5(a) shows the architecture of *Pruner*, which contains a task scheduler (*pSched*) and a set of pruning processing elements (P-PEs). The *pSched* is designed to assign the pruning workloads to the P-PEs. Each P-PE returns pruned results by computing the *smoothness metric* by features of two vertices. A single P-PE is implemented by an adder tree, a MUX, a DMUX, several registers, and ALUs. For example, given an $edge_{(u,v)}$, Fig. 5(a) illustrates the execution process of the single P-PE: ① When the pruning starts, each *Src reg* and *Dst reg* in P-PE read one dimension feature of the vertex u and v from the aggregation buffer, and the *Addr reg* caches the address of the v . ② Multiple ALUs are exploited to execute subtraction operations in parallel to obtain intermediate results. ③ The adder tree collects the intermediate results, and performs the accumulation operation in parallel to generate the temporary smoothness. ④ The DMUX is utilized to determine whether all dimensions have been processed (0: false, 1: true). If DMUX outputs 0, the *Temp reg* is used to store the temporary smoothness, and repeat steps ①, ②, and ③ until all dimensions are executed. ⑤ The feature smoothness and threshold are sent to an ALU to perform comparison operations, and generate the binary bit control state (0: greater than the threshold, 1: less than the threshold). ⑥ Finally, the control state is sent to the MUX to control whether the edge participates in subsequent propagation. If the control status is 1, the address cached by the *Addr reg* is pushed to the *Work FIFO* for subsequent aggregation.

This execution mode assigns a single feature dimension of each vertex to the ALUs in the P-PE, so that multiple dimensions can be processed in parallel, which takes full advantage of the intra-vertex parallelism. When the vertex feature dimension is greater than the dimension that the adder tree in P-PE can handle, we use *Temp reg* to store intermediate results. In this manner, the input vertex properties can be continuously injected into the P-PE, thus enabling P-PE to handle vertices with arbitrary dimensional properties.

C. Design of Aggregator

The aggregation phase needs to collect information based on the edge information. Generally, there are two possible processing modes: vertex-concentrated and vertex-disperse mode. The vertex-concentrated mode assigns the workload of each vertex to a single PE. However, the irregularity of the graph causes the degree of vertices to be inconsistent, and fast vertices must wait for slow vertices, which leads to a long processing delay for a single vertex and further causes the imbalanced workload issue.

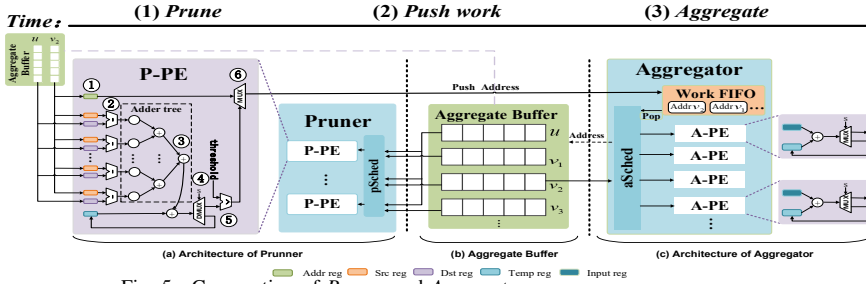


Fig. 5. Cooperation of Pruner and Aggregator.

Therefore, we adopt the vertex-disperse mode, as shown in Fig. 6. In each clock cycle, the aggregating scheduler (*aSched*) fetches a neighbor according to the address. Each aggregate processing element (A-PE) is responsible for aggregating the single dimension property. The *Input reg* in each A-PE sequentially reads the one-dimensional features of the fetched data and performs the feature accumulation, and the accumulated intermediate result is stored in the *Temp reg*. Until all neighbors of the vertices are processed, the *Temp reg* would be cleared. If the dimension of a vertex cannot occupy all the A-PEs, the free A-PEs can be allocated to other vertices. In this manner, the PE group can handle vertex with arbitrary dimension property. Furthermore, all A-PEs are always busy without unbalanced workloads.

D. Design of Updater

The update operations, whether *EdgeUpdate* or *VertexUpdate*, utilize Multilayer Perceptron (MLP) to update vertex features. We adopt the classic systolic array [6] to construct two *Updaters*. As shown in Fig. 7, the *Updaters* can configure the execution mode by the *switch S5* to support the two main categories of GNNs.

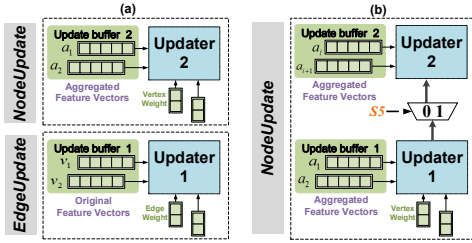


Fig. 7. Execution modes of Updaters. (a) Execution mode for GNNs with *EdgeUpdate*. (b) Execution mode for GNNs without *EdgeUpdate*.

Execution Mode for GNNs with *EdgeUpdate*. As illustrated in Fig. 7(a), the *Updaters* work independently to support different update operations. In this case, *Updater 1* performs *EdgeUpdate*, and *Updater 2* executes *VertexUpdate*. With this mode, the weight parameters are shared among both the *Updaters* and only reused within modules.

Execution Mode for GNNs without *EdgeUpdate*. In this case, we utilize the *switch S5* to assemble two *Updaters* together to form a whole large updater to perform *VertexUpdate*, as shown in Fig. 7(b). In this manner, the weight parameters can flow from *Updater 1* to *Updater 2*, and all *Updaters* can reuse the weight parameters.

With the execution modes, these *Updaters* are always busy, and are not restricted by the GNN's execution flow. This effectively suppresses the issue of idle computing resources and improves resource utilization.

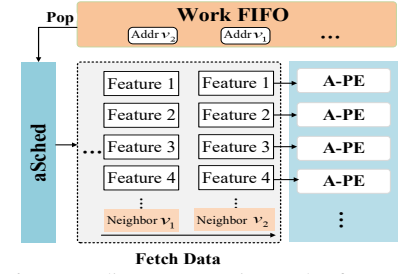


Fig. 6. Vertex-disperse processing mode of Aggregator.

IV. EVALUATION RESULTS

A. Experimental Setup

Methodology. We implement a cycle-accurate simulator to measure the performance of DyGNN accelerator. To measure power and area, we implement each module in Verilog. We synthesize the DyGNN using the Synopsys Design Compiler (DC) with the TSMC 12 nm standard VT library, and estimate power using Synopsys PrimeTime PX. The area and power of on-chip memory are calculated by CACTI [13]. **Evaluated GNN Models and Dataset.** To evaluate DyGNN, we utilize two most popular GNN models: (1) GCN [8], which is GNN without *EdgeUpdate*. (2) GAT [14], which is GNN with *EdgeUpdate*. Tab. V provides the dataset statistics.

Table V. Dataset Statistics.

Dataset	#Vertex	Feature Length	#Edge	Label
Cora (CA)	2,708	1,433	10,556	7
Citeseer (CS)	3,327	3,703	9,104	6
Pubmed (PB)	19,717	500	88,648	3
Reddit (RD)	232,965	602	114,615,892	41

Compared Baselines. We compare DyGNN with two different baselines. The first category is PyTorch Geometric implemented on general-purpose processors, termed as PyG-CPU and PyG-GPU. CPU platform is equipped with two Intel Xeon(R) CPU E5-2680 v3 CPUs and 500GB DRAM, and GPU platform is equipped with NVIDIA Tesla V100. The second category is the state-of-the-art GNN accelerators: HyGCN [17] and HaGNN [1]. HyGCN is designed only for GCN, while HaGNN is designed for general GNNs. For a fair comparison, we enhance the configurations of HaGNN. More detailed configurations are listed in Tab. VI.

Table VI. System Configurations.

	HyGCN	HaGNN	DyGNN
	1GHz @ 32 SIMD 16 cores and 32 × 128 arrays	1GHz @ 512 32-bit ALUs and 32 × 128 PEs	1GHz @ 512 A-PEs, 2 × 16 P-PEs with radix-64 float tree adder and 2 systolic modules (each with 32 × 128 arrays)
On-chip Memory	22MB+128KB	24MB	2MB (Edge), 2 MB (Weight), 4 MB (Output), 2 × 4 MB (Update), and 4 MB (Aggregate)
Off-chip Memory	256GB/s HBM 1.0	256GB/s HBM 1.0	256GB/s HBM 1.0

B. Overall Experimental Results

Algorithm Validation. Tab. VII shows the accuracy and pruning statistics of DyGNN with 4 message passing layers. DyGNN improves the accuracy under different layers. Due to space limitation, these results are omitted. First, we can observe that DyGNN can prune 38.9% edges and 25.4% vertices on average, while improving the accuracy by 4% on average. Second, it shows that DyGNN is also superior to the traditional sampling methods, whether it is sampling vertex (Sv) [2] or sampling edges (Se) [5]. It reveals that the dynamic pruning algorithm can reduce vertex redundancy and

edge redundancy in GNNs and effectively improve the quality of the message passing mechanism. Third, for the large dataset Reddit, original GCN and GAT on PyTorch Geometric cause out-of-memory (OOM). DyGNN can process this dataset and obtain better performance than sampling methods. It shows that DyGNN has good scalability. Note that, the message passing operations are not needed for the pruned vertices. Therefore, *EdgeUpdate* operations are not required to be conducted for the neighbors of the pruned vertices.

Table VII. Accuracy and Pruning Statistics. Sv: the method of sampling vertices, Se: the method of sampling edges, Pv: the ratio of prune vertices, Pe: the ratio of prune edges.

Method	Dataset	Accuracy (%)				Pv (%)	Pe (%)
		Original	Sv	Se	DyGNN		
GCN	Cora	67.1	70.3	66.9	73.8	23.9	33.4
	Citeseer	64.9	56.2	55.2	64.8	28.7	27.7
	Pubmed	76.6	76.6	74.3	78.6	22.7	44.1
	Reddit	OOM	95.0	95.4	96.1	25.3	38.2
	-	66.2	74.5	72.9	78.3	25.2	35.9
GAT	Cora	80.1	80.1	80.2	81.7	22.6	31.4
	Citeseer	66.1	66.4	65.4	68.1	30.3	35.1
	Pubmed	76.1	76.3	76.2	77.7	23.5	59.3
	Reddit	OOM	95.2	96.0	96.4	26.0	42.1
	-	74.1	79.5	79.4	80.9	25.6	42.0

Speedup. Fig. 8 depicts the performance of DyGNN compared to other baselines. Compared with PyG-CPU and PyG-GPU, DyGNN achieves average $815\times$ and $6.2\times$ speedup. It proves that DyGNN can fully exploit the inter-phase parallelism, and can improve the performance by using dynamic pruning mechanism and the pipelined architecture.

DyGNN achieves average speedup by $1.78\times$ on GCN compared with HyGCN, $1.67\times$ on GCN compared with HaGNN and $2.21\times$ on GAT compared with HaGNN. It demonstrates that although the *Pruners* introduce additional computing, it is trivial compared to the improvement. Moreover, the latency time introduced by additional computing can be decreased by the pipelined architecture and the proposed duplication-free pruning scheme. Most importantly, the pruned edges and vertices are not considered in following message passing layers, which not only reduces redundant computing of engines, but also saves off-chip memory access cost. Obviously, DyGNN can achieve better speedup if more GNNs layers are stacked.

We can also observe that the speedup of GAT is better than GCN on DyGNN. An underlying reason is that additional *EdgeUpdate* operations are required to be performed for GAT. Therefore, the computation-intensive MVM operations in *EdgeUpdate* can be significantly reduced in GAT, in addition to the decreased aggregation operations.

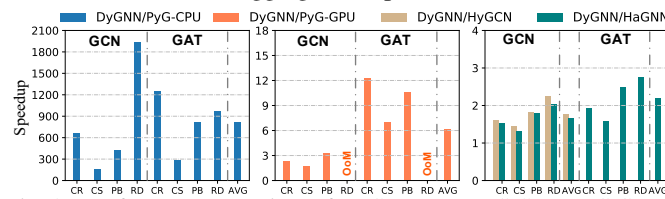


Fig. 8. Performance comparison of DyGNN over PyG-CPU, PyG-GPU, HyGCN and HaGNN.

Energy Consumption. Fig. 9(a) shows the normalized energy efficiency comparison between DyGNN and other baselines. DyGNN achieves energy efficiency by $12.1\times$, $2.3\times$, and $2.5\times$ on average compared to PyG-GPU, HyGCN, and HaGNN, respectively. The energy efficiency is mainly attributed to the fact that DyGNN can greatly reduce the power hungry off-chip memory accesses and save HBM bandwidth.

Area Overhead Breakdown. Fig. 9(b) provides the area breakdown of the execution flow. We can observe that DyGNN executes the *EdgePrune* and *VertexPrune* and doing so increases the overhead. Nevertheless, the area overhead of *Pruners* is much smaller than that of *Updaters*. Although some area overhead is introduced, the computational overhead of other engines will be significantly reduced, especially the *Updater* with numerous MVMs.

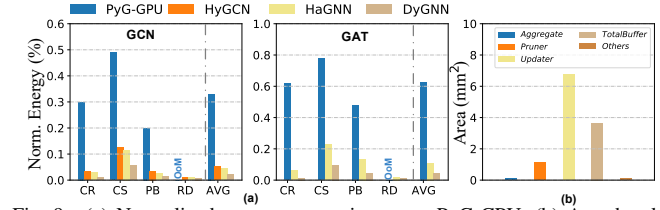


Fig. 9. (a) Normalized energy comparison over PyG-CPU. (b) Area breakdown of DyGNN.

V. CONCLUSION

This paper presents an algorithm and architecture co-design system DyGNN for GNNs. The proposed algorithm can dynamically prune nodes and edges during execution. We also design a novel GNN architecture to support the dynamic algorithm and transform it to performance improvement. We expect that the proposed techniques will contribute to building highly efficient GNNs accelerators for large-scale graphs.

REFERENCES

- [1] A. Auten *et al.*, “Hardware acceleration of graph neural networks,” in *DAC*. IEEE, 2020, pp. 1–6.
- [2] J. Chen *et al.*, “Fastgcn: Fast learning with graph convolutional networks via importance sampling,” in *ICLR*, 2018.
- [3] M. Fey and J. E. Lenssen, “Fast graph representation learning with pytorch geometric,” *arXiv preprint arXiv:1903.02428*, 2019.
- [4] T. Geng, A. Li, R. Shi, C. Wu, T. Wang, Y. Li, P. Haghi, A. Tumeo, S. Che, S. Reinhardt *et al.*, “Awb-gcn: A graph convolutional network accelerator with runtime workload rebalancing,” in *MICRO*. IEEE, 2020, pp. 922–936.
- [5] W. Hamilton *et al.*, “Inductive representation learning on large graphs,” in *NIPS*, 2017, pp. 1024–1034.
- [6] N. P. Jouppi *et al.*, “In-datacenter performance analysis of a tensor processing unit,” in *ISCA*, 2017, pp. 1–12.
- [7] K. Kinningham *et al.*, “Grip: A graph neural network accelerator architecture,” *arXiv preprint arXiv:2007.13828*, 2020.
- [8] T. N. Kipf *et al.*, “Semi-supervised classification with graph convolutional networks,” in *ICLR*, 2017.
- [9] Q. Li *et al.*, “Deeper insights into graph convolutional networks for semi-supervised learning,” in *AAAI*, 2018, pp. 3538–3545.
- [10] S. Liang *et al.*, “Engn: A high-throughput and energy-efficient accelerator for large graph neural networks,” *IEEE TC*, 2020.
- [11] M. Liu *et al.*, “Towards deeper graph neural networks,” in *ACM SIGKDD*, 2020, pp. 338–348.
- [12] L. Ma *et al.*, “Neugraph: parallel deep neural network computation on large graphs,” in *USENIX ATC*, 2019, pp. 443–458.
- [13] T. Shyamkumar *et al.*, “Cacti 5.0,” 2007.
- [14] P. Veličković *et al.*, “Graph Attention Networks,” in *ICLR*, 2018.
- [15] Z. Wu *et al.*, “A comprehensive survey on graph neural networks,” *IEEE TNNLS*, 2020.
- [16] K. Xu *et al.*, “Representation learning on graphs with jumping knowledge networks,” *ICML*, 2018.
- [17] M. Yan *et al.*, “Hygcn: A gcn accelerator with hybrid architecture,” in *HPCA*. IEEE, 2020, pp. 15–29.